LOGAN as a Model and Countermodel Builder

Bounded Adversarial Synthesis for Finite Mathematical Structures

Mirco A. Mannucci
HoloMathics LLC
mirco@holomathics.com

Draft: June 7, 2026

Abstract

Most current machine assistance for mathematics is syntactic: it proposes statements, searches for proofs, or interacts with proof assistants. This paper develops a complementary model-side workflow. We show how LOGAN—Logical GANs based on bounded logical games—can be used to build concrete finite models and countermodels. A Builder proposes a finite structure; an Opponent probes it with bounded logical challenges; and a Judge records either a model certificate or a counterexample certificate. The primitive output is not only a yes/no answer but a small witness: an associativity triple, an identity failure, an inverse failure, a commutativity failure, an EF transcript, or another replayable object. The first implemented arena is finite group theory via Cayley-table synthesis. We give the formal interface, a witness-guided CEGIS loop, examples of models and countermodels, and a companion sub-repository layout with tests. The resulting system is intended as the model-building flank of AI-assisted mathematics: not replacing proof assistants, but supplying finite witnesses, sanity checks, semantic profiles, and counterexamples before proof search hardens a conjecture into a theorem.

Contents

1 Introduction

The standard story of AI-assisted mathematics is still heavily proof-centered. A language model proposes a lemma; a proof assistant checks a derivation; a human decides whether the result is meaningful. This is powerful, but incomplete. Mathematicians do not only prove. They also build examples, test definitions, search for pathological cases, inspect finite shadows, and ask whether a theory has the intended models.

This paper develops the model-building side of LOGAN. The central idea is simple:

> A mathematical object is accepted, for a given budget, if it survives the right bounded adversarial probes; a false conjecture is rejected by producing a small, replayable countermodel witness.

The workflow has three roles.

- Builder proposes a finite structure: a Cayley table, graph, finite algebra, arithmetic fragment, or partial model.
- Opponent probes the object with depth-bounded logical challenges.
- Judge emits a certificate: model survived, model failed with witnesses, or counterexample to a proposed claim.

The conceptual ancestor is the original LOGAN framework, where a bounded logical observer is implemented through Ehrenfeucht–Fraïssé style games and returns interpretable witnesses. The present paper turns that idea toward mathematical construction: groups, graphs, arithmetic fragments, and eventually finite categories or other algebraic gadgets.

The first showcase: finite groups. The companion sub-repository implements a first arena: finite group theory via Cayley tables. The Builder can construct cyclic groups, the Klein four group, and dihedral groups. The Opponent checks associativity, identity, inverses, and optional commutativity. The Judge can certify that a table is a group or produce a counterexample certificate to the claim that all finite groups are abelian.

What this is not. The goal is not to claim that a bounded finite search proves a general theorem. The goal is to add a missing semantic instrument to the mathematical workflow: a system that constructs, attacks, repairs, and explains finite models and countermodels with explicit witnesses.

2 The LOGAN Model-Building Interface

2.1 Structures, theories, and attack surfaces

Fix a finite signature Σ. In the first implementation, $\Sigma = \{\cdot, e, ^{-1}\}$ for group theory. More generally Σ may contain relation symbols, function symbols, constants, or typed operations.

Definition 2.1 (Finite candidate structure)**.** *A finite candidate Σ-structure A consists of a finite domain $|A|$, interpretations of relation symbols as finite tables, and interpretations of function symbols as total or partial finite maps. In the group-theory case, A is represented by a Cayley table*

$$m_A : |A| \times |A| \to |A|$$

with a distinguished identity candidate e_A.

Definition 2.2 (Attack surface)**.** *Given a theory T and a depth parameter k, an attack surface $\mathcal{C}_k(T)$ is a finite or effectively enumerable collection of bounded challenges. Each challenge asks whether a bounded instance of an axiom, property, or comparison game is satisfied by A.*

For universal Horn theories, a challenge is usually a tuple. For example, group associativity is attacked by choosing (x, y, z) and checking

$$(x \cdot y) \cdot z = x \cdot (y \cdot z).$$

If the equation fails, the tuple itself is a witness.

2.2 Witnesses

Definition 2.3 (LOGAN witness)**.** *A* LOGAN *witness is a small, replayable object certifying a failed challenge. Formally it is a tuple*

$$w = (\mathsf{kind}, \mathsf{payload}, \mathsf{message}),$$

where kind *names the failure class,* payload *contains the data needed to replay the failure, and* message *gives a human-readable explanation.*

Example 2.1 (Associativity witness)**.** *For a Cayley table m, an associativity witness has the form*

$$w = (\mathsf{assoc}, x, y, z, m(m(x, y), z), m(x, m(y, z))).$$

It is valid exactly when the two reported values differ.

Example 2.2 (Commutativity counterexample)**.** *To refute "all groups are abelian", it is enough to provide a group A and a pair (x, y) such that*

$$x \cdot y \neq y \cdot x.$$

The counterexample certificate must also record that A passed the group axioms.

2.3 The LOGAN game for model building

Definition 2.4 (LOGAN model-building game)**.** *A position is a tuple*

$$(A, \mathsf{Cert}, \mathsf{Obl}, t),$$

where A is the current finite or partial structure, Cert *is a store of validated facts,* Obl *is a store of existential obligations still to be discharged, and t is the used budget. In each round:*

1. Opponent *chooses a challenge $c \in \mathcal{C}_k(T)$, subject to total budget b.*
2. *If c fails,* Opponent *returns a witness w.*
3. Builder *either repairs A, extends A, or concedes failure.*
4. Judge *updates certificates and logs the trace.*

Definition 2.5 (Bounded LOGAN model)**.** *We write*

$$A \Vdash_{k,b} T$$

when Builder *has a strategy that survives all* Opponent *challenges from $\mathcal{C}_k(T)$ up to budget b, while preserving its certificate discipline.*

This is deliberately weaker than classical satisfaction. It is an operational notion: A survived a specified adversarial attack surface.

3 Model Certificates and Counterexample Certificates

3.1 Model certificates

A model certificate records:

- the structure name and representation;
- the theory or property suite challenged;
- the depth and budget used;
- all witness classes checked;
- the fact that no failing witness was found.

In the finite group setting, a cyclic group C_n is certified by checking associativity, identity, and inverses over its Cayley table. For a total table the check can be exhaustive; for a larger or partial structure it can be sampled or budgeted.

3.2 Counterexample certificates

A counterexample certificate to a claim

$$T \Rightarrow \varphi$$

records:

- a structure A;

- a certificate that A satisfies or survives the assumptions T;
- a witness w showing that φ fails in A.

For example, to refute "all finite groups are abelian", LOGAN returns a dihedral group D_3 of order six, verifies the group axioms, then returns a pair (x, y) witnessing non-commutativity.

4 CEGIS: Witness-Guided Synthesis

The implementation follows a CounterExample-Guided Inductive Synthesis pattern. The difference is that the counterexamples are mathematical witnesses rather than ordinary test failures.

Algorithm 1 LOGAN ModelBuilder

Require: theory T, depth k, budget b, builder policy π, opponent policy δ

initialize candidate structure A_0
initialize $\mathsf{Cert} \leftarrow \varnothing$, $\mathsf{Obl} \leftarrow \varnothing$, trace $\tau \leftarrow []$
for iteration $i = 1, \ldots, N$ **do**
    $W, D \leftarrow \mathsf{Opponent}_\delta(A_i, T, k, b)$
    if $W = \varnothing$ and $D = \varnothing$ **then**
        return model certificate for A_i
    end if
    append W, D to trace τ
    $A_{i+1} \leftarrow \mathsf{Builder}_\pi(A_i, W, D, \mathsf{Cert})$
    update Cert with replayable validated facts
end for
return failure or unknown with trace τ

5 Finite Groups as the First LOGAN Arena

5.1 Signature and axioms

The group signature is

$$\Sigma_G = \{\cdot, e, {}^{-1}\}.$$

The first implementation represents the operation by a total Cayley table and treats inverses semantically: an element x has an inverse when some y satisfies

$$x \cdot y = e = y \cdot x.$$

The attacked axioms are:

$$\begin{aligned} (x \cdot y) \cdot z &= x \cdot (y \cdot z), \\ e \cdot x &= x = x \cdot e, \\ \exists y\, (x \cdot y = e \wedge y \cdot x = e). & \end{aligned}$$

Optionally, the $\mathsf{Opponent}$ also attacks commutativity:

$$x \cdot y = y \cdot x.$$

This is not a group axiom; it is a target property used to build counterexamples.

5.2 Implemented builders

The P0 sub-repository includes deterministic builders for:

- cyclic groups C_n, represented by addition modulo n;
- the Klein four group V_4, represented by $\mathbb{Z}_2 \times \mathbb{Z}_2$;
- dihedral groups D_m, represented as pairs (i, j) with multiplication

$$(i, j)(k, \ell) = \big(i + (-1)^j k \bmod m,\ j + \ell \bmod 2\big).$$

5.3 Implemented opponents

The P0 opponent is a Horn-style finite group opponent. It checks the current structure and returns the first replayable witnesses it finds:

- `associativity_failure`;
- `identity_failure`;
- `inverse_failure`;
- `commutativity_failure`, when commutativity is part of the challenge.

Later versions should make the budget operational by sampling instances, prioritizing hard tuples, and mixing Horn checks with EF probes.

5.4 Showcase 1: building a model

Command:

```
logan-modelbuilder build-group --kind cyclic --n 5
```

Expected status:


```
{
  "status": "survived",
  "structure": "C_5",
  "order": 5,
  "witnesses": []
}
```


Interpretation: C_5 survives the group-theory attack surface. In a full paper result table, this becomes a model certificate row.

5.5 Showcase 2: building a counterexample

Command:

```
logan-modelbuilder counterexample --claim all_groups_abelian --max-order 8
```

Expected status:

```
{
  "claim": "all_groups_abelian",
  "structure_name": "D_3",
  "structure_order": 6,
  "assumptions_verified": true,
  "refuting_witness": {
    "kind": "commutativity_failure",
    "payload": {"x": ..., "y": ..., "x*y": ..., "y*x": ...}
  }
}
```

Interpretation: LOGAN has not merely guessed that the theorem is false. It has produced a finite group plus a replayable witness to the negation of the target property.

6 Repo Architecture

The companion sub-repository is meant to be integrated into the existing LOGAN repo, but it is packaged so that it can also run independently.

```
logan_modelbuilder/
  pyproject.toml
  README.md
  src/logical_gans/modelbuilder/
    structures.py
    witnesses.py
    engine.py
    cli.py
    theories/
      groups.py
    opponents/
      horn_group.py
    builders/
      group_builder.py
  experiments/
    e1_groups.py
  tests/
    test_groups.py
  scripts/
    run_all.sh
```

Design choice. The namespace `logical_gans.modelbuilder` is intentional: this should feel like a LOGAN companion module, not an unrelated new package.

7 Experimental Plan

7.1 P0 experiments

Experiment	Structure	Target	Expected result	Witness type
E1.1	C_n	group axioms	survives	none
E1.2	V_4	group axioms + abelian	survives	none
E1.3	D_3	group axioms	survives	none
E1.4	D_3	abelian	fails	commutativity pair
E1.5	random table	group axioms	fails	assoc/id/inverse

7.2 P1 experiments

The next layer should include:

- monotone fill of partial Cayley tables;
- local revision with edit costs;
- certificate-preserving repair;
- random and SAT/SMT baselines;
- JSONL traces of witness/repair/certificate events.

7.3 Metrics

For each run, record:

- pass/fail/unknown;
- number and class of witnesses;
- repair cost;
- certificate growth;
- wall-clock time;
- minimal budget at which a target property fails.

8 Relation to UCMT

This paper should be read as the implementation-facing companion to the UCMT line. UCMT says that bounded modelhood can be formulated as survival against bounded logical challenges. LOGAN ModelBuilder supplies the computational discipline:

- concrete finite structures;
- explicit Opponents;
- small witnesses;

- model and countermodel certificates;
- reproducible traces.

The deeper thesis is that model theory can become operational. A theory is not only a collection of sentences. It determines an attack surface. A model-building system is not only a search routine. It is an adversarial dialogue between construction and falsification.

9 Conclusion

LOGAN ModelBuilder is a model-side mathematical assistant. It does not replace theorem provers; it supplies the finite semantic evidence that should often come before proof search. The first group-theory implementation is deliberately modest but already captures the essential pattern: build a structure, attack it, return witnesses, and distinguish model certificates from counterexample certificates. The next step is to make the builder genuinely repair-oriented, extend the opponent from Horn checks to EF probes, and grow the domain library from groups to monoids, graphs, arithmetic fragments, and finite categories.

A Claude Development Checklist

The companion file `CLAUDE_ACTION_LIST.md` gives the implementation checklist. The highest-priority tasks are: integrate into the existing LOGAN tree, make budgeted challenge sampling real, add partial Cayley-table repair, and generate result tables for the paper.

References

[1] A. Ehrenfeucht. An application of games to the completeness problem for formalized theories. *Fundamenta Mathematicae*, 49(2):129–141, 1961.

[2] R. Fraïssé. Sur quelques classifications des systèmes de relations. *Publications Scientifiques de l'Université d'Alger*, 1:35–182, 1954.

[3] M. A. Mannucci. Logical GANs: Adversarial learning through Ehrenfeucht–Fraïssé games. Draft / preprint, 2025.

[4] M. A. Mannucci. Ultraconstructive Model Theory via Logic driven Adversarial Synthesis. Draft, 2026.

[5] R. Alur, R. Singh, D. Fisman, and A. Solar-Lezama. Search-based program synthesis. *Communications of the ACM*, 61(12):84–93, 2018.